NETWORK INTRUSION DETECTION COMPARISON STUDY

CSCC11 Final Project Submitted by:

Joshua Antonio Crisologo

Albert Hyunh

Daniel Venistan

Nethanel Woubishet Reta

Table of Contents

1 Introduction **1**

1.1 Problem Statement . . . 1

1.2 Existing Approaches and Our Direction . . . 1

2 Data Preprocessing **2**

2.1 Dataset Description . . . 2

2.2 Data Exploration . . . 2

2.3 Data Cleaning and Feature Engineering . . . 2

2.4 Data Representation and Splitting . . . 2

3 Model Comparison Study **2**

3.1 Model Descriptions and Motivations . . . 2

3.1.1 Logistic Regression . . . 2

3.1.2 Multi-Layer Perceptron (MLP) . . . 3

3.1.3 Random Forest . . . 3

3.1.4 XGBoost . . . 3

3.2 Evaluation Setup . . . 4

3.3 Model and Tuning Summary . . . 4

3.4 Main Results . . . 4

4 Originality and Future Work **5**

4.1 Originality . . . 5

4.2 Future Work . . . 5

A Appendix **6**

A.1 Exploratory Plots . . . 6

A.2 Per-Attack Category Tables . . . 6

A.3 Feature Breakdown . . . 6

A.4 Categorical Frequency Tables . . . 6

References **13**

ABSTRACT

This project compares four supervised models for binary network intrusion detection on the UNSW-NB15 dataset: Logistic Regression, Multi-Layer Perceptron (MLP), Random Forest, and XGBoost. After duplicate removal, grouping rare categorical values, applying log transformations to 14 highly skewed numerical features, and building model-specific tabular representations with 58 predictors, we trained on 107,740 flows and evaluated on a held-out test set of 55,945 flows. Hyperparameters were tuned with RandomizedSearchCV using 3-fold cross-validation and ROC-AUC, and final classification thresholds were chosen by sweeping from 0.01 to 0.99 to maximize attack-class F1. All models improved after threshold tuning: Logistic Regression reached a tuned F1 of 0.7811, MLP 0.8571, Random Forest 0.8784, and XGBoost 0.8680. XGBoost achieved the best CV ROC-AUC among tree-based models (0.9601) and is selected as the final model; Random Forest achieved the highest test ROC-AUC (0.9729) and tuned F1 (0.8784). Per-attack-category analysis showed that Fuzzers was the hardest attack family across all models, while Worms, Generic, and Backdoor were detected with near-perfect recall by the tree-based models. Overall, the comparison shows that nonlinear ensemble methods are best suited to this tabular intrusion-detection task, and XGBoost provides the best balance of detection quality and generalization.

Introduction

The purpose of the project is to analyze the applications of machine learning models in the field of Cyber Security. One rising application of ML within this industry is to detect malicious network activity. This can come in the form of cyber attacks and intrusions. Training an ML model to detect these attacks is essential to improving the efficiency of cybersecurity systems [1]. We aim to utilize ML to detect anomalies and/or cyber attacks through network traffic.

1.1 Problem Statement

In this project, we study the UNSW-NB15 network intrusion dataset from Kaggle [2]. This dataset represents simulated network traffic as tabular data. Each row is a network record that is either labelled as normal traffic or a cyber attack, representing a binary classification problem. Attacks can also be labelled into multiple categories, such as Fuzzers, Analysis, Backdoors, etc. Tabular data with labels is well suited for an ML classification problem.

Thus, our task is a binary classification problem: given a network record, we want to predict whether it corresponds to normal traffic (label 0) or an attack (label 1). Our main research question is:

How do different ML models compare on modern tabular network intrusion detection data, and which model is the most robust and scalable in a cyber security setting?

Using logistic regression as a baseline, we want to see how tree-based models (Random Forest, XGBoost) and neural networks compare to our baseline and to each other, and which model provides the most robustness in a cyber security setting, balancing both robustness in predicting attacks and scalability in inference speed and large data.

1.2 Existing Approaches and Our Direction

Existing ML approaches to network intrusion detection include fitting tree-based models or deep learning on a processed dataset. Some use simpler models such as Random Forest while others use more novel models such as XGBoost, CatBoost, or LightGBM, or even neural networks depending on the dataset. Most existing work evaluates models using a fixed decision threshold of 0.5 and reports overall accuracy or F1, without breaking down performance across attack categories or tuning the threshold to the specific trade-offs of the intrusion detection setting.

Our method is to compare four models on the same binary classification task under a shared preprocessing and evaluation pipeline: Logistic Regression, Multi-Layer Perceptron, Random Forest, and XGBoost. Logistic Regression serves as the simple linear baseline, while Random Forest provides a baseline for a standard tree-based model. MLP is used to showcase the power of neural networks on tabular data and to see if it can outperform tree-based models, while XGBoost is our advanced method. We chose this model because it extends the course ideas and our learning about decision trees, random forests, gradient descent, and ensemble models into a stronger, tree-based ensemble using gradient-boosted trees. Additionally, we wanted to work with XGBoost, as it is known for being one of the strongest and most scalable models for tabular data, making it a well-motivated advanced model for our project.

Data Preprocessing

2.1 Dataset Description

Our dataset, as explained in Section 1.1, is the UNSW-NB15 dataset [2]. This dataset contains records representing network traffic labelled as either normal or attack and has 49 columns, such as `sbytes`, representing transaction bytes from source to destination or `sloss`, being dropped source packets. The full data dictionary can be found under `data/raw/data_dict_UNSW-NB15_features.csv`. Our task is binary classfication, to predict whether a record in normal (0) or an attack (1). The dataset is pre-split into a training set of 107,740 records and a test set of 55,945 records after duplicate removal.

2.2 Data Exploration

In our training set, our records are split 48.2% normal (51,890) and 51.8% attack (55,850). We have a nearly balanced training set, however, the test set is less balanced, having 61.1% normal records (34,206) and 38.9% attack (21,739). In our EDA, we found 14 heavily right-skewed numerical features (such as `sload, sbytes`) having ranges up to 10^9. Three categorical features had a large amount of unique values (`proto, service, state`), as well as 11 pairs with high correlation ($|r| > 0.9$). Figures A.1 - A.3 in the appendix plot the raw numerical data and the log transformations we applied, as well as the categorical frequency plots, and the correlation heatmap.

2.3 Data Cleaning and Feature Engineering

To prevent data leakage, we removed duplicate rows from both training and test sets before preprocessing. We also removed the columns `id` (row identifier), `attack_cat` (the multi-class label), and `stcpb/dtcpb` (random sequence numbers). All categorical features were one-hot encoded and condensed by putting categories with rare values into an `other` category, converting nominal categories into numeric inputs that models can use. We also applied a log transformation ($log(1+x)$) on our skewed numerical features to make the data more normal. Numerical features were standardized using `StandardScaler` for logistic regression and MLP models so that gradient-based optimization converges faster on normalized inputs, while the tree-based models use the raw values as they are invariant to scaling.

2.4 Data Representation and Splitting

Two datasets are derived from our preprocessing pipeline: `(training)/(test)_lr_mlp` (scaled numericals, used by Logistic Regression and MLP) and `(training)/(test)_xgb_rf` (raw numericals, used by Random Forest and XGBoost), which both have 58 columns plus the label. Preprocessing methods such as the `StandardScaler` are saved with `joblib`, which are used to process the test set. For all models, hyperparameter tuning uses `RandomizedSearchCV` with 3-fold cross-validation on the training set, with evaluation on the test set.

Model Comparison Study

3.1 Model Descriptions and Motivations

3.1.1 Logistic Regression

Logistic regression is a linear model that uses a linear combination of its inputs to model the log-odds of the positive class and outputting a probability using the sigmoid function. More specifically, it uses the formula $p = \sigma(w^T x + b)$ to calculate these odds, and is optimized by minimizing cross-entropy loss using gradient descent. To prevent overfitting, the model also supports L1, L2, and elastic-net regularization (L1 sparsifies coefficients, L2 shrinks them, and elastic net combines both). It is a popular baseline for classification problems, especially on tabular data due to its simplicity, accurate predictions, and interpretability.

We chose the logistic regression model as we want to use it as a baseline for predictive power. Since it is one of the simplest models, it is the best reference for judging if the more complex models perform well. We believe this model would fit well on our dataset due to its tabular data with some correlated features, where logistic regression's optimization and regularization shine.

3.1.2 Multi-Layer Perceptron (MLP)

Multi-Layer Perceptron (MLP) is a supervised learning algorithm and is also known as a modern feedforward neural network. It consists of fully connected dense layers that transform input data from one dimension to another. These dense layers are made up of neurons with nonlinear activation functions, which allow the network to model complex patterns and distinguish data that is not linearly separable, and can be used for both classification or regression.

The basic workflow of the MLP can be simplified via forward propagation, loss functions, backpropagation and optimization. Each neuron in a hidden or output layer computes a weighted sum of its inputs: $z = \sum_i w_i x_i + b$ where x_i is the input value, w_i is the corresponding weight, and b is the bias term. After calculating z, the neuron applies an activation function to produce its output. When we reach the output layer, the model's error is calculated using a loss function — binary cross-entropy for classification, or mean squared error (MSE) for regression. Once the loss is computed, we calculate the gradient of the loss with respect to each weight, which is then propagated backward through the network layer by layer, allowing us to update the weights in a way that reduces the loss in future predictions. To prevent overfitting, MLPs commonly use regularization techniques such as dropout, which randomly deactivates a fraction of neurons during training, and L2 regularization (weight decay), which penalizes large weights in the loss function to encourage simpler, more generalizable models.

We selected the MLP for two reasons: First, MLP benefits directly from the scaled and log-transformed preprocessing applied to our dataset, as gradient-based optimization converges more effectively on normalized inputs, which is an advantage that tree-based models like XGBoost and Random Forest do not require. Second, MLP builds naturally on what we learned in lecture about neural networks and gradient descent, making it a fitting and well-motivated addition to our model comparison.

3.1.3 Random Forest

Random Forest [3] is a tree-based ensemble algorithm that builds a large number of decision trees and, in classification, determines a label using majority vote. Each tree is trained with a sample of the training data (bagging) with only a random subset of features to reduce variance/overfitting, and an ensemble improves prediction power compared to a single tree. Because trees are trained independently in parallel, any form of boosting is not used, which differs from XGBoost. This trades the bias-reduction benefits of boosting for lower sensitivity to noise, since errors are averaged out rather than sequentially built upon.

We selected Random Forest as a tree-based baseline for three main reasons. Firstly, Random Forest is one of the most well-established and widely-used algorithms for tabular classification tasks, which makes it a competitive reference against more complex models (XGBoost, MLP). Secondly, we wanted to evaluate whether boosting (XGBoost) meaningfully outperforms bagging (Random Forest) on this specific dataset and problem setting. Lastly, Random Forest extends our learning from lecture concepts such as decision trees and ensembles, making it a motivated choice along the more advanced models.

3.1.4 XGBoost

XGBoost [4] stands for Extreme Gradient Boosting and is an ML framework that provides a gradient-boosted decision tree (GBDT) implementation that is scalable and distributed. Similar to Random Forest and the topics we learned in class, a GBDT is an ensemble decision tree learning algorithm. RF builds decision trees independently in parallel via bagging, while XGBoost uses gradient boosting to build a new tree that learns from the previous tree's "mistakes." These errors are represented by setting an objective function and minimizing it using the gradient descent algorithm to create the targeted outcome for the next model.

XGBoost extends gradient boosting through its use of both the first and second order gradient statistics (gradient and Hessian) on the loss function to find the optimal score, leading to fast convergence and a better tree structure. The objective is also regularized using both L1 (lasso) and L2 (ridge) regularization in the objective function itself to decrease overfitting. It also takes advantage of shrinkage, which scales new weights by a factor after a step of boosting and thus ensures no one tree has too much influence, and row/column subsampling, which introduces randomness similar to the random forest algorithm. It also uses sparsity-aware splitting to handle missing/sparse features, and a weighted quantile sketch to find efficient splits, making it robust and scalable.

We selected XGBoost as the advanced/novel model in our project for three main reasons. Firstly, XGBoost is widely known as the SOTA ML when it comes to classification problems on tabular data. We wanted to gauge XGBoost's performance on this set of tabular data with a large number of features. Secondly, we wanted to compare tree-based models vs MLP on tabular data (with logistic regression as a baseline). Neural networks are rising in popularity, however, we wanted to show that deep learning approaches may not be the most optimal, as we found that tree-based models outperformed it in a tabular setting. Lastly, XGBoost extends many concepts covered in lecture (decision trees, ensembles, gradient descent), making it a motivated choice.

3.2 Evaluation Setup

All models were trained and evaluated on the same train-test split, using 3-fold cross validation for finding the optimal hyperparameters. **Metrics:** ROC-AUC was used as the metric for cross-validation as it measures the model's ability to rank positives above negatives (handling imbalance) independent of a threshold. Precision, recall, and F1 were then reported on the test set because a robust model must balance false alarms against missed attacks. Accuracy would be misleading due to the slight imbalance in training set and moderate imbalance in the test set. **Tuning:** Model hyperparameters were tuned with RandomizedSearchCV using search spaces described in Table 3.1 that cover both simple and expressive settings without becoming computationally intense. Our hyperparameter choices are reasonable as these spaces span from underfitting to overfitting areas (low to high c, shallow to deep trees, etc.), and our sweep finds the most reasonable choice. **Threshold analysis:** After fitting the best cross-validated model, thresholds from 0.01 to 0.99 were swept and the threshold maximizing attack-class F1 was selected. Our goal, which differs from the typical goals other literature of purely maximizing F1 score or accuracy, was to see the optimal threshold that would maximize each model's ability to distinguish between false and real attacks. The default 0.5 threshold may not be optimal as the model may miss attacks due to being conservative. The tuned predictions were also joined back to the raw `attack_cat` labels to find out which attack category models struggle to classify.

3.3 Model and Tuning Summary

Table 3.1: Hyperparameter search spaces and best configurations for each model.

Model	Hyperparameters tried and best configuration
Logistic Regression	Tried `C` on a log-uniform range from 1e-3 to 1e3; `penalty` values `l1`, `l2`, and `elasticnet`; `class_weight` values `None` and `balanced`; `l1_ratio` from 0.1 to 0.9; `solver` = `saga`; `max_iter` = 2000. Best: `penalty` = `elasticnet`, `C` = 897.9856, `l1_ratio` = 0.3434, `class_weight` = `None`.
MLP	Tried `hidden_layer_sizes` from single- to multi-layer layouts between (32,) and (256,128,64,32); `activation` values `relu` and `tanh`; `learning_rate_init` values 1e-2, 1e-3, 5e-4, 3e-4, and 1e-4; `alpha` values 1e-4, 1e-3, 1e-2, 0.1, and 1.0; `batch_size` values 32, 64, 128, and 256. Best: `hidden_layer_sizes` = (128,64), `activation` = `relu`, `learning_rate_init` = 0.01, `alpha` = 0.001, `batch_size` = 256.
Random Forest	Tried `n_estimators` values 100, 200, 300, and 500; `max_depth` values None, 10, 20, 30, and 40; `min_samples_split` values 2, 5, 10, and 20; `min_samples_leaf` values 1, 2, 4, and 8; `max_features` values `sqrt`, `log2`, 0.3, and 0.5; `bootstrap` values True and False. Best: `n_estimators` = 300, `max_depth` = 10, `min_samples_split` = 20, `min_samples_leaf` = 4, `max_features` = 0.3, `bootstrap` = True.
XGBoost	Tried `n_estimators` values 200, 300, 500, 700, and 1000; `max_depth` values 3, 4, 5, 6, and 8; `learning_rate` values 0.01, 0.05, 0.1, and 0.2; `subsample` and `colsample_bytree` values 0.6, 0.7, 0.8, and 1.0; `min_child_weight` values 1, 3, and 5; `gamma` values 0, 0.1, 0.3, and 0.5; `reg_alpha` values 0, 0.1, 0.5, and 1.0; `reg_lambda` values 0.5, 1.0, 2.0, and 5.0. Best: `n_estimators` = 1000, `max_depth` = 4, `learning_rate` = 0.01, `subsample` = 0.7, `colsample_bytree` = 0.8, `min_child_weight` = 1, `gamma` = 0.1, `reg_alpha` = 1.0, `reg_lambda` = 2.0.

3.4 Main Results

Table 3.2: Cross-model comparison on validation and test metrics.

Model	CV ROC-AUC	Test ROC-AUC	Default F1	Best Threshold	Tuned F1	Takeaway
Logistic Regression	0.9435	0.9217	0.7507	0.76	0.7811	Weakest test ROC-AUC and tuned F1; Fuzzers hardest category at 0.4618 recall.
MLP	0.9884	0.9578	0.7982	0.82	0.8571	High CV ROC-AUC but lower test score suggests mild overfitting; Fuzzers hardest at 0.4694 recall.
Random Forest	0.9484	0.9729	0.8062	0.74	0.8784	Best test ROC-AUC; highest tuned F1 across all models.
XGBoost	0.9601	0.9695	0.8175	0.69	0.8680	Best CV ROC-AUC among tree models; selected as final model.

All four models improved after threshold tuning compared to the default 0.50 threshold. Logistic Regression improved from 0.7507 to 0.7811, MLP from 0.7982 to 0.8571, Random Forest from 0.8062 to 0.8784, and XGBoost from 0.8175 to 0.8680. This confirms that threshold tuning produces meaningfully better operating points for all models. The tree-based models show close CV–test alignment (XGBoost: CV 0.9601 vs. test 0.9695; RF: CV 0.9484 vs. test 0.9729), indicating well-chosen search spaces. MLP shows a larger gap (CV 0.9884 vs. test 0.9578), suggesting mild overfitting on the cross-validation folds.

XGBoost achieved the best CV ROC-AUC among tree-based models (0.9601) and is **selected as the final model**, as CV ROC-AUC was the primary optimization metric and XGBoost's CV–test gap is small. Random Forest achieved the highest test ROC-AUC (0.9729) and highest tuned F1 (0.8784), making it the strongest alternative. Logistic Regression produced the weakest test ROC-AUC (0.9217) and tuned F1 (0.7811), confirming that a linear boundary is insufficient for this dataset. Both tree-based models substantially outperformed LR and MLP on test ROC-AUC, showing the tabular features contain nonlinear structure that ensemble trees exploit more effectively.

Per-attack-category analysis shows Fuzzers as the hardest category across all four models: Logistic Regression achieved 0.4618 recall, MLP 0.4694, and both XGBoost and Random Forest 0.6255 — reflecting the randomized nature of fuzzing traffic which closely resembles normal flows. The tree-based models achieved 0.88 recall or higher on all other categories, with XGBoost reaching perfect recall (1.0000) on Worms. MLP performed well on Analysis (0.9955 recall) and most categories above 0.93, while Logistic Regression showed consistent but lower recall across all categories.

Originality and Future Work

4.1 Originality

Most studies that compare models on network intrusion detection datasets use a default classification threshold of 0.5. For example, More et al. [5] evaluates classifiers on UNSW-NB15 using fixed thresholds and accuracy, without per-category analysis or threshold tuning. In this project, we adopt a different approach by tuning the classification threshold for each model rather than fixing it. As explained in Section 3, we test values from 0.01 to 0.99 and pick the one that gives the best F1 score, then compare those results to the standard 0.5 threshold.

The reason for this is that in cybersecurity applications, missing an attack (a false negative) is usually far worse than raising a false alarm. By adjusting the threshold according to this trade-off, we end up with models that are more useful in real-world scenarios rather than benchmarking accuracy.

Instead of only looking at overall metrics, we break down how each model performs across the nine attack categories in the UNSW-NB15 dataset using the `attack_cat` feature. Kasongo and Sun [6] similarly study UNSW-NB15 but focus on feature selection and overall accuracy without breaking down performance per attack category. Our per-category analysis gives a clearer picture of each model's strengths and weaknesses. For example, Fuzzers was the hardest attack type to detect for all models.

4.2 Future Work

One possible direction for future work would be to replace the scikit-learn `MLPClassifier` with a deep learning framework such as PyTorch or Keras. Unlike scikit-learn's MLP, these frameworks support dropout, batch normalization, and per-class loss weighting, enabling custom loss functions such as focal loss that penalise hard-to-classify samples like Fuzzers more heavily, which directly addresses the weakest category across all models in this study.

Another promising direction is model ensembling, stacking, or experimenting with novel models. In this study, Logistic Regression, Random Forest, XGBoost, and MLP were each evaluated independently, yet each model showed different strengths across attack categories. For example, tree-based models excelled on other categories except for Fuzzers. Rather than selecting a single best model, a stacking approach could combine the predictions of all four models, allowing the ensemble to exploit the complementary strengths of each classifier. Experimentation with other novel models such as CatBoost, LightGBM, or other deep learning architecture may result in an improvement in performance metrics.

Finally, all models in this study were trained and evaluated exclusively on the UNSW-NB15 dataset, which was collected in a lab environment in 2015. This raises questions about how well these models would generalize to real-world traffic or more recent datasets such as CIC-IDS2017. Since network traffic and attack patterns evolve over time, cross-dataset validation would be a valuable next step to better assess the true generalizability of these models.

Appendix

A.1 Exploratory Plots

This appendix collects the supplementary exploratory figures referenced in the data chapter.

A.2 Per-Attack Category Tables

The following figures reproduce the per-attack-category table outputs reported in each model notebook.

Per-attack precision is uniformly 1.0 by construction of this breakdown. Each table groups only true attack rows after excluding Normal traffic, so no false-positive cases remain within an attack-category subset. As a result, recall and F1 are the more informative per-category metrics.

A.3 Feature Breakdown

Table A.1: Raw categorical and numerical features in the original dataset.

Feature Type	Variables
Categorical (3)	`proto`, `service`, `state`
Numerical (37)	`ackdat`, `ct_dst_ltm`, `ct_dst_sport_ltm`, `ct_dst_src_ltm`, `ct_flw_http_mthd`, `ct_ftp_cmd`, `ct_src_dport_ltm`, `ct_src_ltm`, `ct_srv_dst`, `ct_srv_src`, `ct_state_ttl`, `dbytes`, `dinpkt`, `djit`, `dload`, `dloss`, `dmean`, `dpkts`, `dttl`, `dur`, `dwin`, `is_ftp_login`, `is_sm_ips_ports`, `rate`, `response_body_len`, `sbytes`, `sinpkt`, `sjit`, `sload`, `sloss`, `smean`, `spkts`, `sttl`, `swin`, `synack`, `tcprtt`, `trans_depth`

A.4 Categorical Frequency Tables

Table A.2: Retained categorical levels based on training-split frequencies.

Feature	Retained levels (counts)	Collapsed into `other`
`proto`	`tcp` (43,095), `udp` (29,418), `unas` (3,515), `arp` (987), `ospf` (676), `sctp` (324)	4,317
`service`	`-` (47,153), `dns` (21,367), `http` (8,287), `smtp` (1,851), `ftp` (1,552), `ftp-data` (1,396), `pop3` (423), `ssh` (204)	99
`state`	`FIN` (39,339), `INT` (34,163), `CON` (6,982), `REQ` (1,842)	6

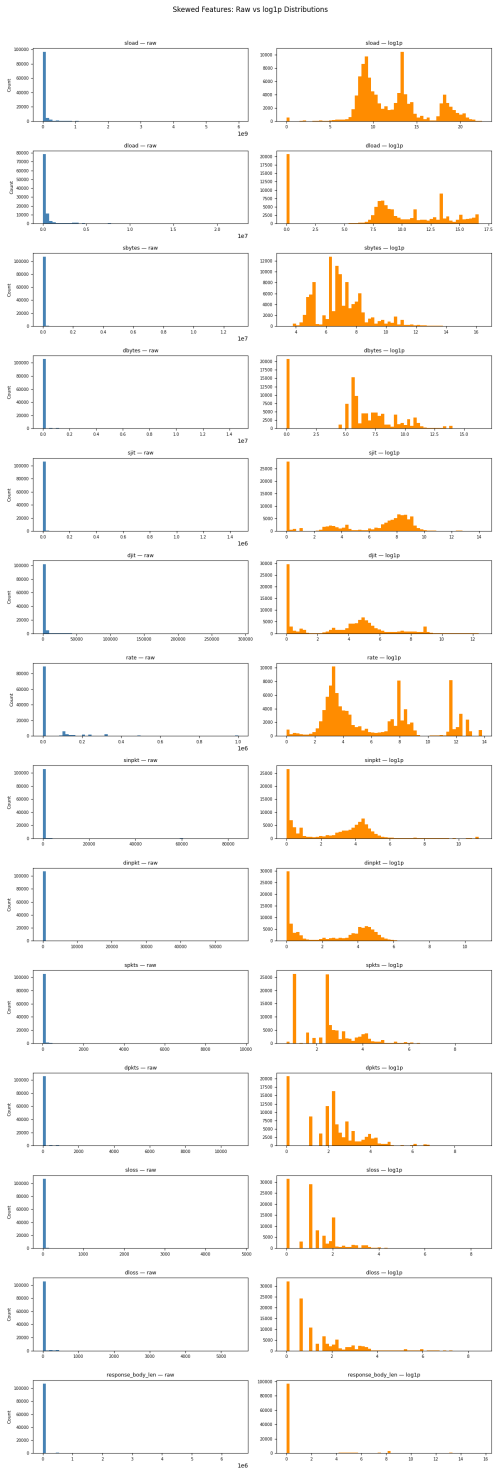

Figure A.1: Raw versus $\log(1+x)$ distributions for the heavily skewed numerical features identified during exploratory analysis.

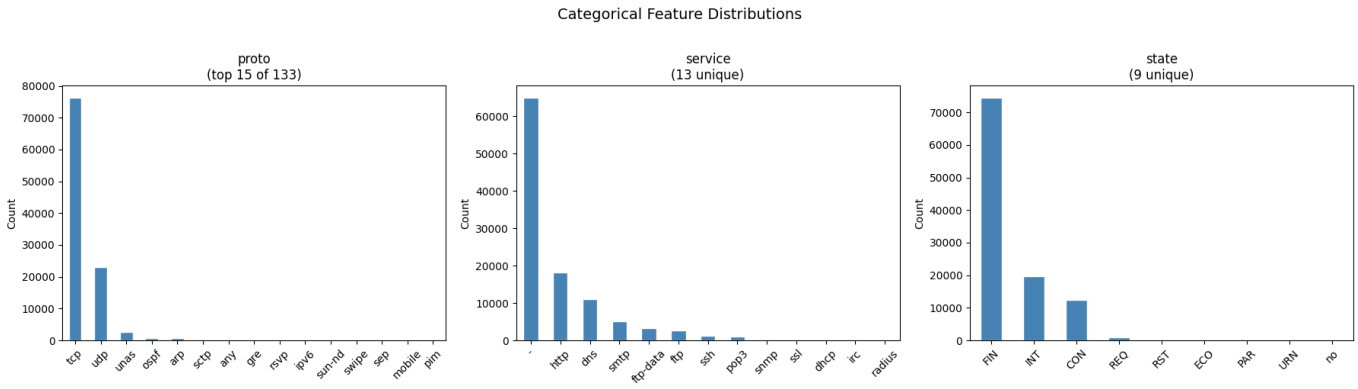

Figure A.2: Frequency distributions of the bucketed categorical features used in the final preprocessing pipeline.

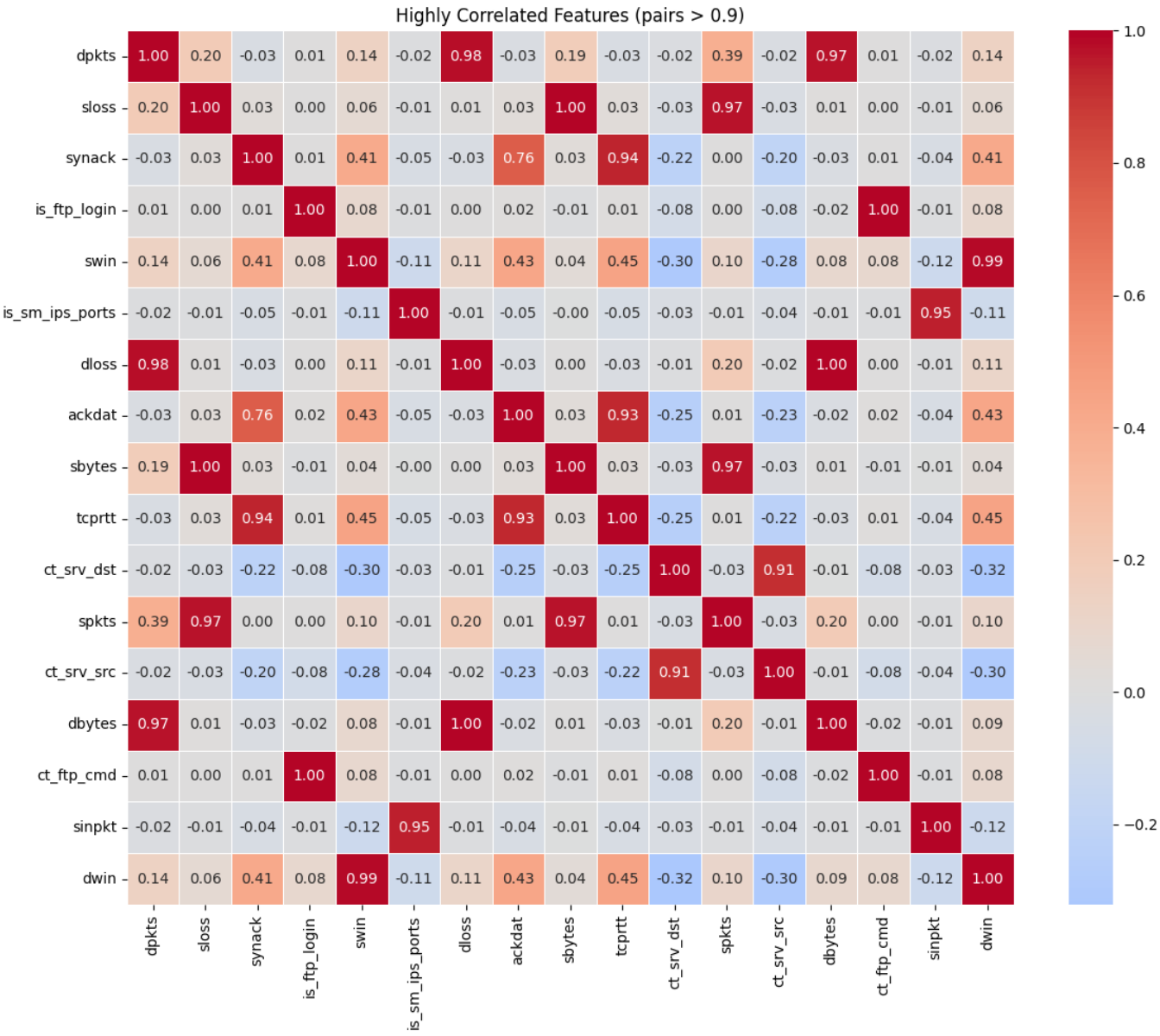

Figure A.3: Correlation heatmap over the numerical features used during exploratory analysis.

attack_cat	count	precision	recall	f1
Analysis	1594.0	1.0	0.9586	0.9789
Fuzzers	16150.0	1.0	0.9591	0.9791
Worms	127.0	1.0	0.9685	0.9840
DoS	3806.0	1.0	0.9727	0.9861
Backdoor	1535.0	1.0	0.9752	0.9875
Reconnaissance	7522.0	1.0	0.9829	0.9914
Exploits	19844.0	1.0	0.9843	0.9921
Shellcode	1091.0	1.0	0.9853	0.9926
Generic	4181.0	1.0	0.9969	0.9984

Figure A.4: Logistic Regression per-attack-category table output.

Figure A.5: Logistic Regression per-attack-category bar chart.

attack_cat	count	precision	recall	f1
Analysis	1594.0	1.0	0.8350	0.9101
Fuzzers	16150.0	1.0	0.8906	0.9421
Shellcode	1091.0	1.0	0.9743	0.9870
Reconnaissance	7522.0	1.0	0.9795	0.9897
DoS	3806.0	1.0	0.9877	0.9938
Exploits	19844.0	1.0	0.9898	0.9949
Backdoor	1535.0	1.0	0.9941	0.9971
Generic	4181.0	1.0	0.9969	0.9984
Worms	127.0	1.0	1.0000	1.0000

Figure A.6: MLP per-attack-category table output.

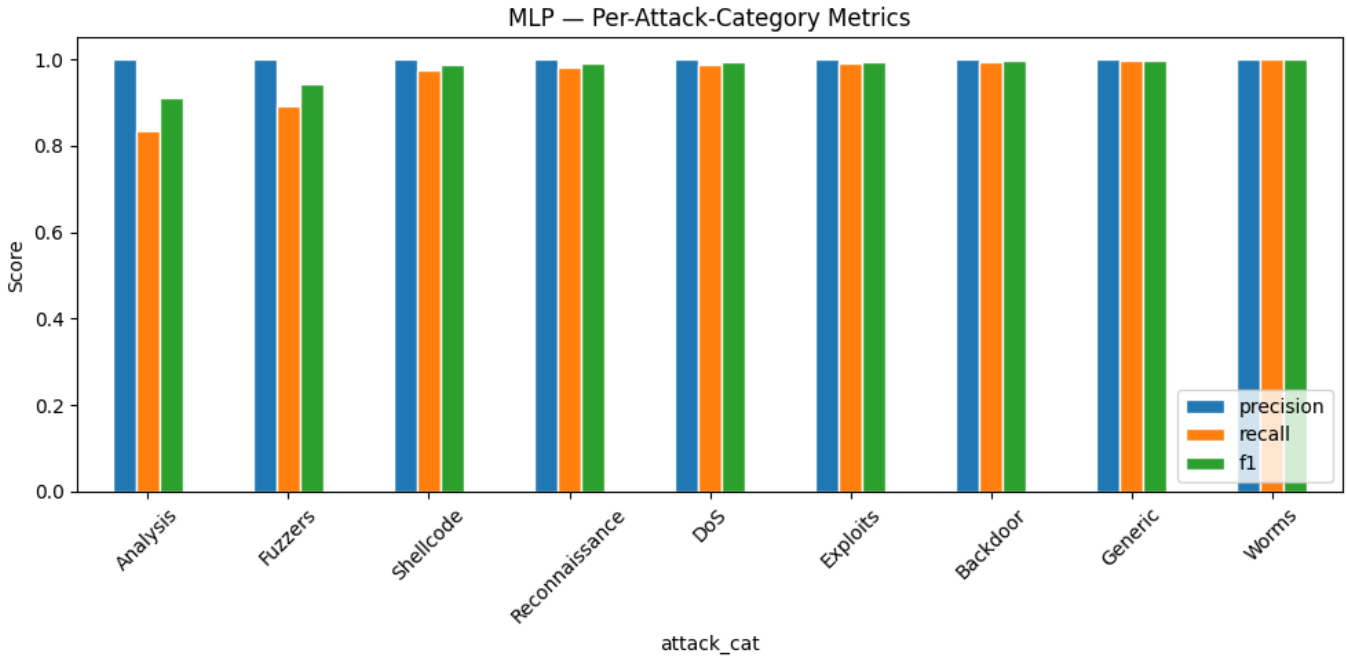

Figure A.7: MLP per-attack-category bar chart.

attack_cat	count	precision	recall	f1
Fuzzers	4838.0	1.0	0.6255	0.7696
Analysis	446.0	1.0	0.9462	0.9724
Shellcode	378.0	1.0	0.9762	0.9880
Worms	44.0	1.0	0.9773	0.9885
Exploits	7609.0	1.0	0.9787	0.9892
Reconnaissance	2703.0	1.0	0.9834	0.9916
DoS	1718.0	1.0	0.9837	0.9918
Backdoor	346.0	1.0	0.9913	0.9956
Generic	3657.0	1.0	0.9964	0.9982

Figure A.8: Random Forest per-attack-category table output.

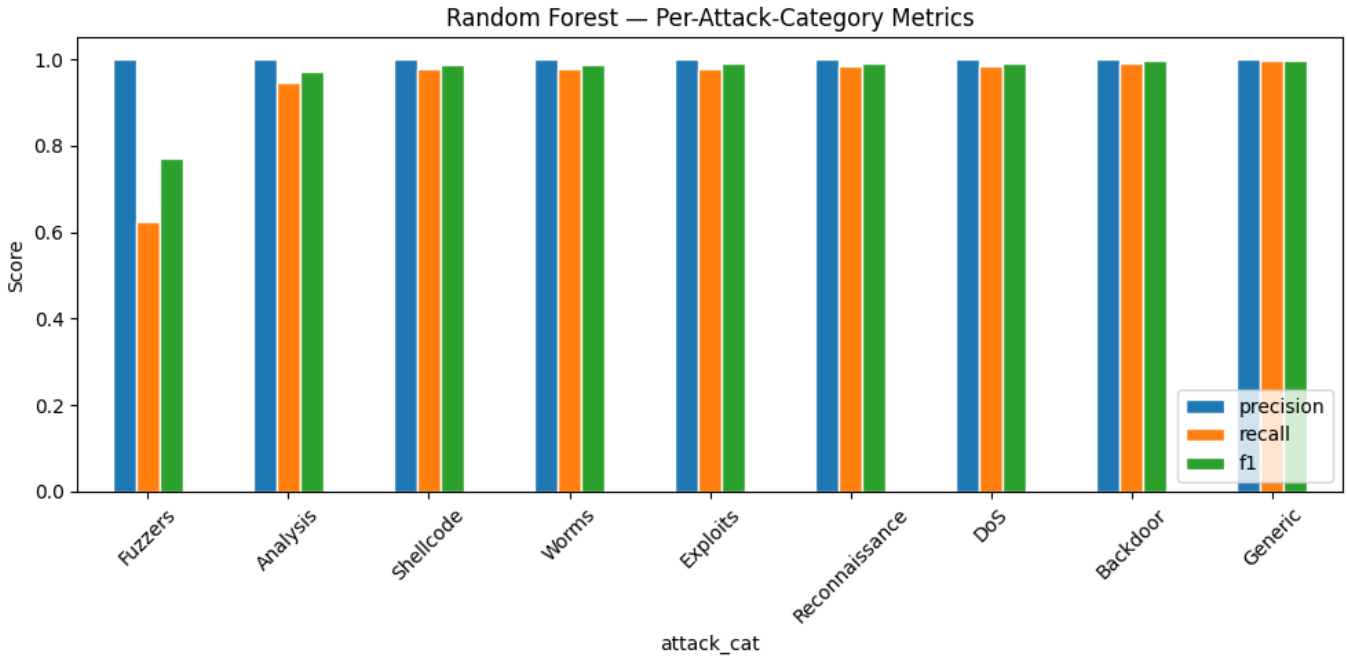

Figure A.9: Random Forest per-attack-category bar chart.

attack_cat	count	precision	recall	f1
Fuzzers	4838.0	1.0	0.6255	0.7696
Analysis	446.0	1.0	0.8812	0.9368
Shellcode	378.0	1.0	0.9656	0.9825
Exploits	7609.0	1.0	0.9823	0.9910
DoS	1718.0	1.0	0.9866	0.9933
Reconnaissance	2703.0	1.0	0.9970	0.9985
Backdoor	346.0	1.0	0.9971	0.9986
Generic	3657.0	1.0	0.9973	0.9986
Worms	44.0	1.0	1.0000	1.0000

Figure A.10: XGBoost per-attack-category table output.

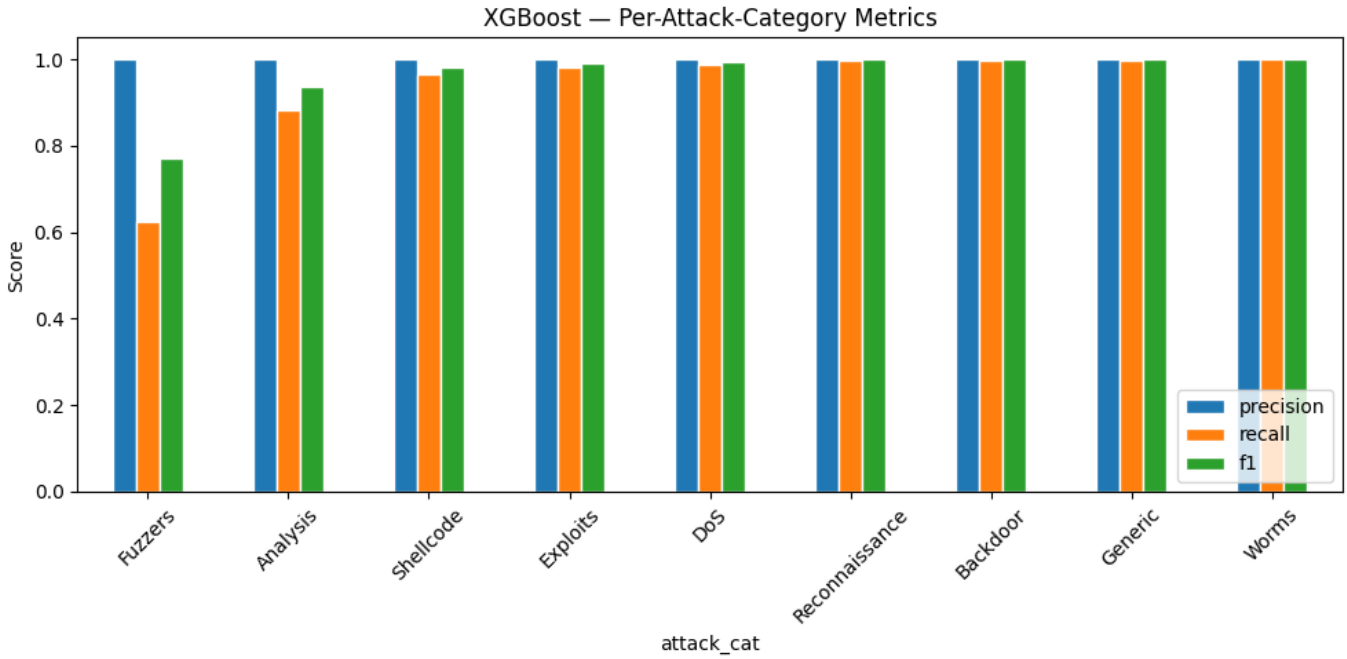

Figure A.11: XGBoost per-attack-category bar chart.

References

[1] Andrea Pinto, Luis-Carlos Herrera, Yezid Donoso, and Jairo A. Gutierrez. Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure. *Sensors*, 23(5):2415, 2023.

[2] David Wells. UNSW_NB15. https://www.kaggle.com/datasets/mrwellsdavid/unsw-nb15, 2018. Kaggle dataset page, accessed 2026-04-04.

[3] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Sanjay More, Mina Idrissi, Hani Mahmoud, and A. Taufiq Asyhari. Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. *Algorithms*, 17(2):64, 2024.

[6] Sydney M. Kasongo and Yanxia Sun. Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *Journal of Big Data*, 7:105, 2020.